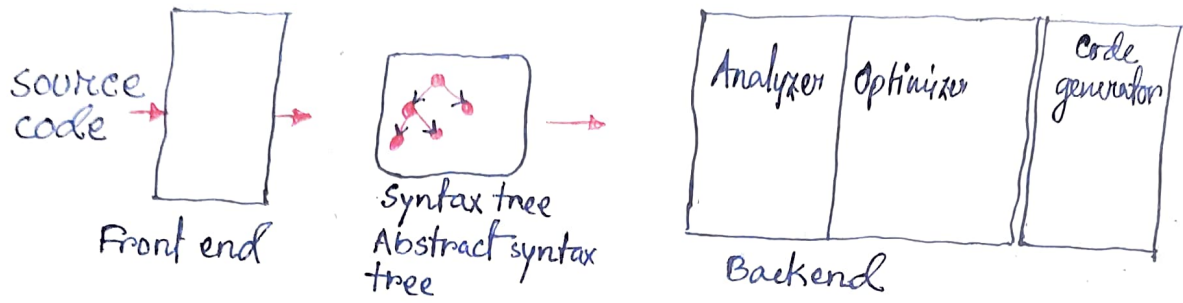
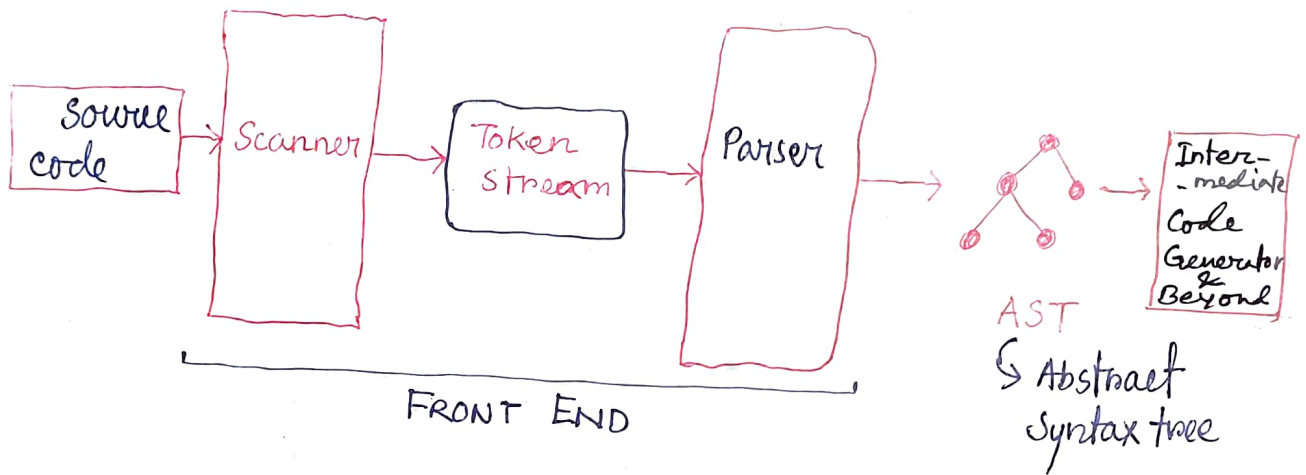


☐ Schematic of Compiler/Interpreter steps : RECAP



- ☐ Front end converts source program text into AST format.



- Scanner/lexer : Converts program source code into tokens, that is, into keywords, variable names operators, number etc.

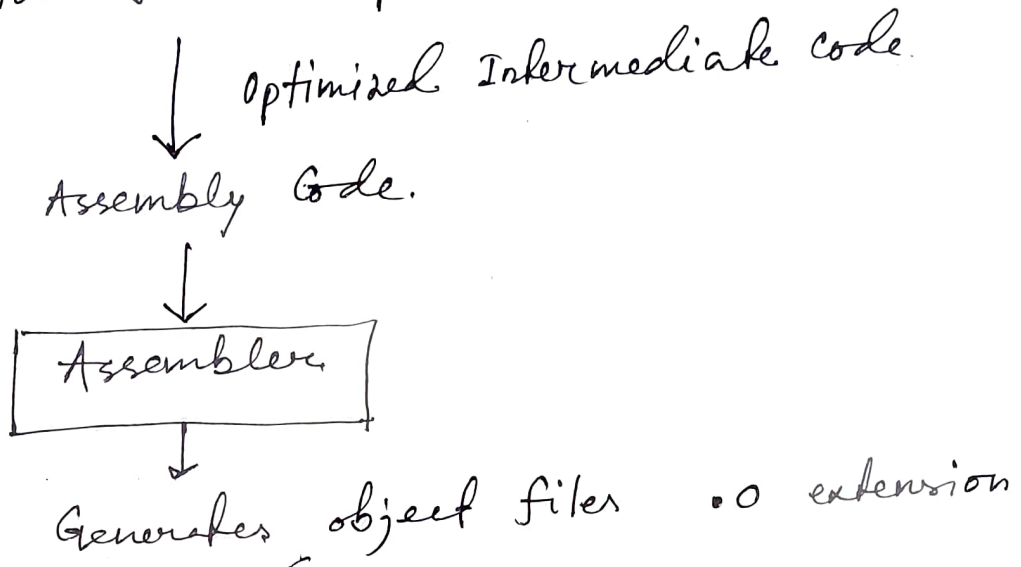
- Parser : Takes tokens and converts those into Abstract Syntax Tree (AST)

↳ Done using context free grammar.

- ☐ AST, means Abstract Syntax Tree, is easier for further use to optimize, analyze and execute the program

After AST is formed, we can perform optimization to eliminate redundancy to produce compact representation of the process ~~is~~ coded in the source code.

The optimized AST is passed on to the Intermediate Code Generator. The IR, generated from the code generator, is subsequently treated for further optimization.



↪ If a process needs multiple .o files along with other pre-compiled libraries, all are used by a linker to produce an executable file. That is .exe file.

So, Linker creates the .exe file.

Linker

A linker is a software tool necessary to produce an executable file.

Linker combines separate parts of a program and generate a self-sufficient executable file.

↳ later loaded in the memory by a loader

Often we write code into different pieces — each piece might have new functions and variables dependent on other parts.

A linker does this connectivity between pieces smoothly.

Example: Role of linker in C/C++. Linker is not a part of compiler but language implementation process should keep it in consideration.

Even if an exact linker does not exist, arrangements implementing such tasks as the dedicated linker does must be available.

Specific Tasks done by a linker

- Combine the different pieces of a program together
- Resolve the memory organization, ~~that~~ often be a new memory organization, so that all the different parts fit together
- Make improvements in addresses so that the program can run as per the new memory organization.

A linker must solve the below

- Suppose "printf" routine — Assembler does not know the location of printf routine when assembling files.

↓
Address of external objects
↓
Unknown to assembler

↓
So, it puts "zero"
for each unknown
address.

- Assembler does not know the destination memory addresses at the time of assembling these files to produce object files.

↓
Assembler sets that everything starts at address zero.

↓ expects
Linker to do the
necessary re-arrangement

object file:

- Generated by the compiler and assembler
- Each source-code file part has separate object file
- Remains incomplete, as one object file may include dependency on the other object file.

What do we have in an object file?

Object file header: Information ~~for~~ of the other pieces of object file.

Text segment: The machine code form of the instructions.

Data segment: Binary representation of the static data in the source file.

relocation information: identifies lines of code that need to

Symbol Table: Contains the list of the labels of this file and static data that can be referenced.

Considering the above content of the object files and that the functioning of linker shown in previous figure, linker steps are

- Take text segments from each separate object file and ~~put~~ include those in the executable file
- Take data segment from each object file and merge them together, and then concatenate the relocated data ~~to~~ after the text.
- Resolve references.

7] so, summarizing the underlying steps associated with compiling a program (for instance, a C-program) provides the below steps:

C-program: example.c

↓
Compiler (does lexical analysis, syntax analysis)

↓
Assembler

↓
Produced Object file: example.o

↓
Linker (interfaces with system files, libraries)

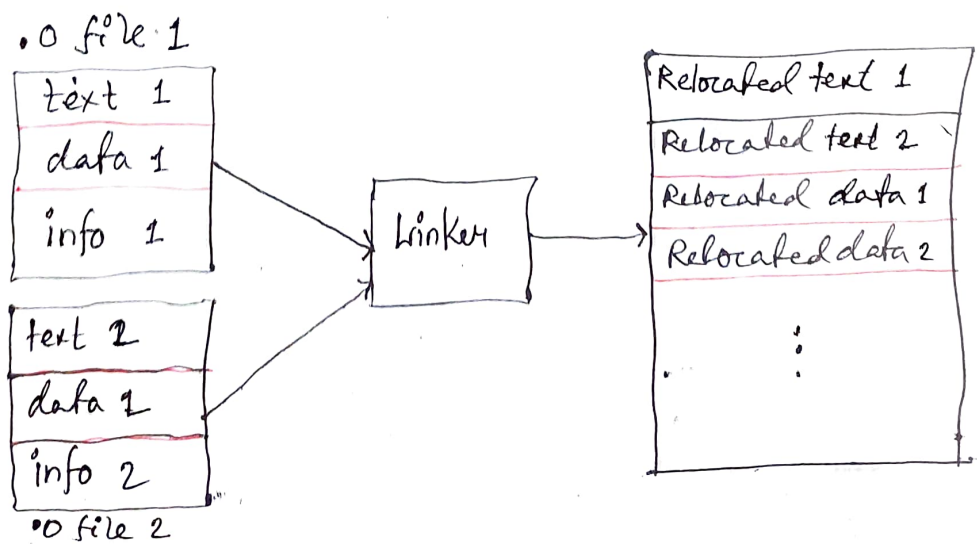
↓
Executable version: example.out

↓
Loader (Part of the Operating System)

↓
Memory

Here, linker

combines object files (.o) to form a single executable

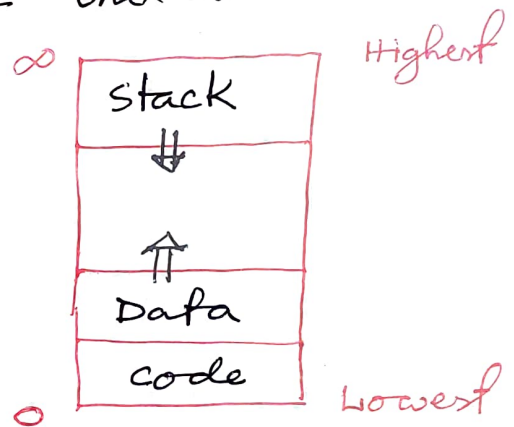


Operating system concepts relevant for program compilation.

Generally, the memory layout for operating system, for instance unix-based system, include

- Code or text : starts at 0 location
- Data : starts immediately above code & moves/grows upward
- Stack : starts at the highest address and grows towards the lower address.

Suppose, we consider the unix-based memory layout schematic:



To run a process (which could be through a compiled program), the memory management is done by system components -

• Compiler and assembler:

• Generate one object file for each source code file containing information

↳ However, the information ~~now~~ is incomplete. Because, each source file can have some references ~~to~~ defined in other files

❏ Linker:

- It links all the object files for a program into one executable file
- It's output is self-sufficient and complete

❏ Operating System:

- Loads executable files into memory
- Ensures multiple processes sharing memory at once.
- Also, provides the necessary arrangements to obtain more memory once a process is already started

❏ Runtime Library:

- Works together with the operating system to provide dynamic allocation routine

Further focus on Linker:

- combine many separate pieces of a program
- Reorganize storage location
- Typically, A Linker is invoked by the compiler and it is done implicitly.

Functions of a linker:

- combine all the separate pieces of a program.
 - ↳ can have multiple source files
 - ↳ Or, source files and the other standard i/p o/p library files as needed
- Generate a new memory organization and performs address rearrangement.

Overall, these roles of a linker finally produces the Runnable program.

↳ stored in another object file
↳ known as
Executable file

Q What sort of transformations done on the addresses by an assembler?

Output of the assembler is of the form of object file and contains information in 1's and 0's.

However, assembler is unaware of the memory locations of things that it is assembling.

How is this memory address issue resolved?

Answer: Linker solves this memory addressing.

So, linker must solve the below

Assembler

→ does not know the memory locations for information it assembles to

→ It is unaware of external objects' addresses at the time of assembling.

Assembler puts zero in the object files.

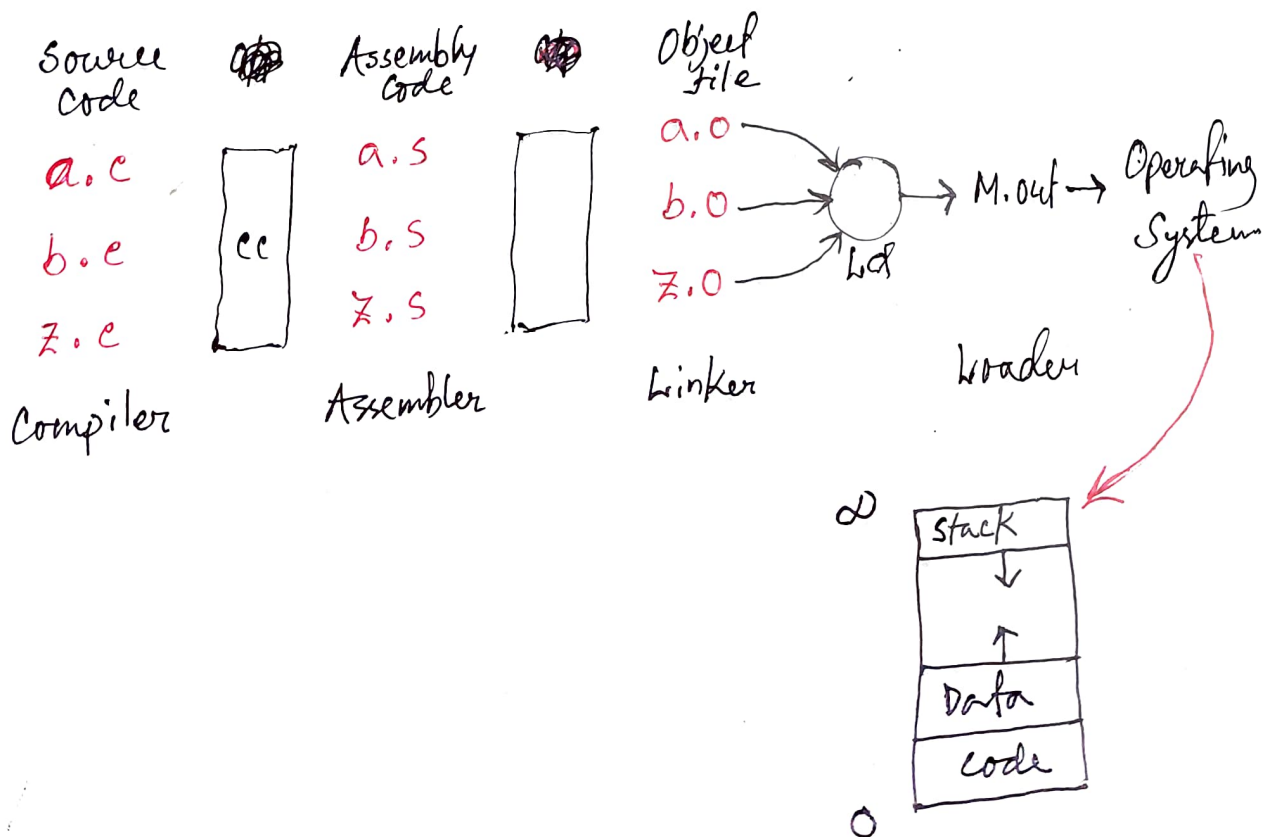
Solution by linker

Assembler assumes each section to be start at address zero, & linker re-arranges all

linker ~~also~~ resolves this by adding offset to the base address. However, it also keep track of size of sections of object files.

So, the ~~memory~~ rearrangement done by a linker goes as follows: *Revising how a process is created at the system level.*

Example:



A sample example: Consider the below c-code.

CS 140 Stanford
Source

```
extern float sin();
extern printf(), scanf();

main () {
    double x, result;
    printf("Type number: ");
    scanf("%f", &x);
    result = sin(x);
    printf("sin is %f\n", result);
}
```

main.c

```
FILE* stdin, stdout;
int printf(const char*
           format, ...) {
    ...
    fputs(c, stdout);
    ...
}

int scanf(const char*
          format, ...) {
    ...
    c = fgetc(stdin);
    ...
}
```

stdio.c

```
double sin(double x) {
    ...
}
```

math.c

Object file of ~~math~~
main.c

main.o

main.o

A

Address	Code	Section
0	main	Text Section
...	...	
30	call printf	
...	...	
52	call scanf	
...	...	
60	call sin	
...	...	
86	call printf	

B

Address	Code	Section
0	-S1: Type number	Data Section
14	-S2: %f	
17	-S3: "sin is %f\n"	
main T[0]		Symbols
-S1:	D[0]	
-S2:	D[14]	
-S3:	D[17]	

printf	T[30]
printf	T[86]
scanf	T[52]
sin	T[60]
-S1	T[24]
-S2	T[54]
-S3	T[80]

store the final location
of sin at an offset 60 in the main
text section.

Overall, a linker executes its stipulated tasks in three passes:

Pass 1: It reads in section size, and computes final memory layout. For instance, a sample output of the Pass 1 phase for the previously stated process would appear as -

Memory map: After Pass 1

⋮	
stdio.o	data
main.o	data
math.o	text
stdio.o	text text
main.o	text text

Pass 2: It reads all symbols and produces a complete symbol table in memory.

Pass 3: It reads in section and relocation information, update ~~addresses~~ addresses, and finally write everything in a new file.

Reference: CS143 Stanford University
CS140 Stanford University